

序列算法与序列问题

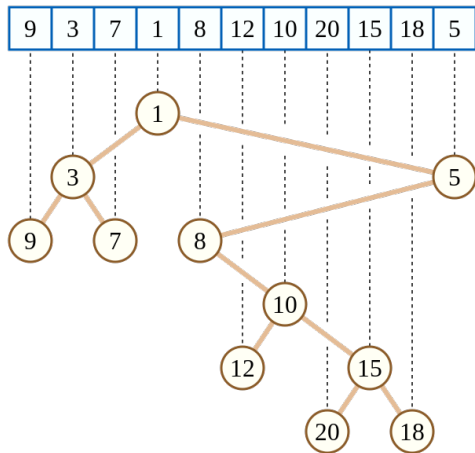
pwq

Cap1taL

2026 年 7 月

?! 笛卡尔树?!

笛卡尔树

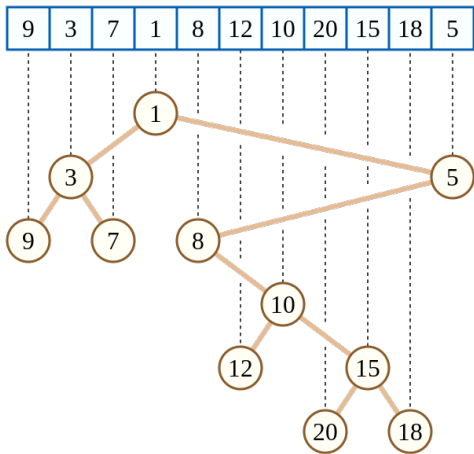


我们默认讨论小根的情况

- 笛卡尔树是一种二叉树。点集为 (i, a_i) 。第一键值满足 BST 的性质，第二键值满足堆的性质。
- 直观来说，根节点为最小值，左右儿子分别为左右两侧的最小值，以此类推
- 运用一些数据结构可以随便在 $O(n \log n)$ 内建出笛卡尔树

?! 笛卡尔树?!

我找了好多好多性质 (1)



- ① 笛卡尔树的一个子树 u 总是一个区间。
- ② 在不改变区间最小值的情况下，一个子树 u 代表的区间是极大的。
- ③ 区间 $[L, R]$ 的最小值是 L, R 在笛卡尔树上的 LCA。
- ④ 随机排列的期望笛卡尔树高是 $O(\log)$ 的。

?! 笛卡尔树?!

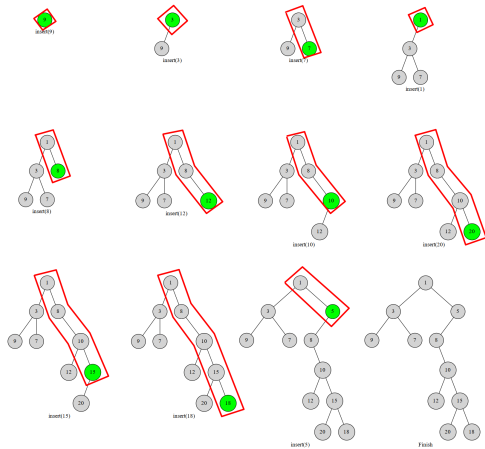
我找了好多好多性质 (2)

```
1 L[i] R[i] 为最近的比a[i]小的数
2 for(int i=1;i<=n;i++){
3     if(i-L[i]<R[i]-i){
4         for(int j=L[i]+1;j<=i;
5             j++){
6             //do smth O(1)
7         }
8     }else{
9         for(int j=i;j<R[i];j
10            ++){
11            //do smth O(1)
12        }
13    }
14 }
```

- 这段代码的复杂度是 $O(n \log n)$
- 为什么?
- hint: 树上启发式合并?

?! 笛卡尔树?!

线性建笛卡尔树



- 维护右链
- 每次插入 u 从下向上比较右链，直到找到 v 使得 $a_v < a_u$
- 让 u 成为 v 的右儿子， v 原本的右儿子成为 u 的左儿子
- 每个点只会进出右链一次

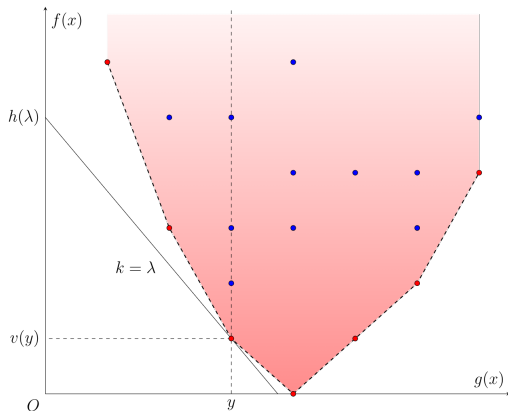
?!WQS 二分?!

WQS 二分的动机

- 有 n 个元素，我们要求选出 k 个元素，最优化一个复杂的目标函数
- 由于目标函数较为复杂（甚至难以刻画），状态转移不得不成为二维的 $F(n, k)$ ，复杂度至少为 $O(nk)$ 。这可能是难以接受的。
- 而无元素个数限制的问题常常是容易考虑的
- 我们能不能每选一个物品就“惩罚”一下算法，再想办法巧妙地选取“惩罚”，使得算法恰好找到我们想要的 $f(n, k)$ 呢？
- 这只是一个粗略的想法。具体如何操作？有什么限制？

?!WQS 二分?!

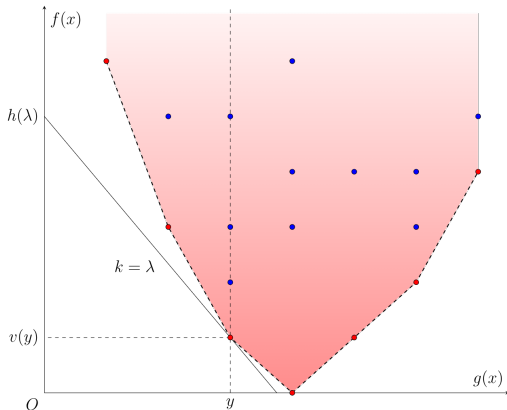
WQS 二分的基本思想



- 凸集，我们要求得某个 (x, y)
- 直接求不好求，尝试用直线去切？
- 为什么可以二分？
- 如何 check？

?!WQS 二分?!

共线情形的处理



- 什么是共线情形？为什么会出现问题？
- 本质上，共线时，我们期望算法最后计算唯一的 $\text{check}(\lambda)$
- 方案一：在 check 中引入单调性，从唯一寻找 (x, y) 变为最优化不等关系。用直线方程代入得出答案
- 方案二：引入实数二分，不断缩小范围到 λ 附近

我分了好多好多治

这一部分的内容主要包括

- 基本分治
- CDQ 分治
- 线段树分治
- 整体二分 (optional)
- 树分治不太相同，我们放到树上算法章节

基本分治的思想

- 一个问题不好处理
- 将一个问题划分为多个（经常为两个）易求解的子问题，然后把问题留给后人，进行递归，最后合并得到整个问题的答案
- 没了
- 说起来非常简单，实际上分治较为灵活，更像是一种思想
- 具体的分治做法与具体题目密切相关

CDQ 分治的思想

- 基本分治有时把问题划分左右两个子问题，最后合并左右的答案
- 如果左右子问题之间相互影响怎么办？
- CDQ 分治的基本思想是将两侧的互相影响贡献补全
- 看具体题目

P1908 逆序对



求序列逆序对个数
 $n \leq 5 \times 10^5$

Solution

- 我们不妨假设这个问题没有 $O(n \log n)$ 的做法 (?)
- 考虑如何合并左右区间
- 我们需要处理跨区间的逆序对
- 先扫描左半侧的所有元素，将它们加入值域数据结构维护
- 扫描右半侧的所有元素，在值域数据结构上查询，得到逆序对数量
- 复杂度 $O(n \log^2 n)$

P3810 【模板】三维偏序 / 陌上花开

有 n 个元素，第 i 个元素有 a_i, b_i, c_i 三个属性，设 $f(i)$ 表示满足 $a_j \leq a_i$ 且 $b_j \leq b_i$ 且 $c_j \leq c_i$ 且 $j \neq i$ 的 j 的数量。

对于所有 $d \in [0, n)$ ，求有多少 $1 \leq i \leq n$ 使得 $f(i) = d$ 。

对于所有数据，保证 $1 \leq n \leq 10^5$ ， $1 \leq a_i, b_i, c_i \leq k \leq 2 \times 10^5$ 。

Solution

- 我们先立刻对 a 排序
- 再考虑对第二维 b 做归并排序。是的，可以打乱 a
- 这不是白干了吗？
- 虽然左右两个子问题内的 a 已经被打乱了，但在我们合并左右问题之前，左侧所有元素的 a 均小于右侧元素的 a
- 若我们只考虑跨中线的元素对，那么 a 第一维限制实际上仍能满足
- 双指针扫描 b 做归并，实时维护一个值域数据结构，把左侧的 c 加入，用右侧的 c 查询
- 复杂度 $O(n \log n \log V)$
- 需要去重后单独考虑重复元素的贡献。
- 刚开始对 a 排序时第二关键字也需要对 b 排序，否则 a 相同时会出问题。

线段树分治能干什么？

- 若存在一些操作，这些操作的“存活时效”是一段连续的区间，且穿插着询问，则我们可以考虑使用线段树分治
 - 例如添加一个元素，删除一个元素，则每个元素的“存活时效”是一段连续的区间
- 线段树分治可以把这样的问题变为 $O(Q \log Q)$ 次的“增加操作”和“撤销上一次的添加”
- 撤销会更好做吗？

线段树分治做了什么？

- 将所有操作和询问离线
- 我们在长度为 Q 的时间轴上建立一颗线段树，在线段树的每个节点上都开一个 vector。
- 若在 T_1 时插入一个元素 E ，在 T_2 时间移除这个元素 E ，它的存活时间即为 $[T_1, T_2]$
- 我们把 $[T_1, T_2]$ 这个区间在线段树上拆成 $O(\log)$ 个区间，在对应的节点的 vector 里“挂载 E ”
- 对于回答询问，我们完整遍历整个线段树
 - 从上向下递归经过一个节点时，我们“处理”节点上“挂载”的所有元素
 - 当访问到了叶子时，我们回答这个时间点对应的询问（若有）
 - 递归结束，向上退出这个节点时，我们不断执行撤销

Thanks!